

Summary Report

1. Project Goal

The goal was to build a C++ project that goes beyond a minimal chess game: a legal chess board plus a neural-network and evaluator visualizer. The final program needed to satisfy course requirements for classes, multi-file C++ structure, file I/O, inheritance, templates, a linked list, tests, design documentation, user documentation, and demo material.

2. Final Result

We built a native C++17/raylib desktop application with:

- complete human-vs-human legal chess play,
- move highlighting, promotion, undo/redo, SAN move history, FEN load/save, and setup-editor validation,
- a modular chess core with bitboards, mailbox board lookup, FEN/SAN support, perft validation, Zobrist hashing, and draw/status helpers,
- four evaluation visualizations: NNUE, LC0 CNN, LC0 BT4, and Classical,
- compact local model files for NNUE/CNN/BT4 runtime demos,
- a native PUCT MCTS search-tree workbench,
- config, FEN, PGN export, binary model, and test-data file I/O,
- automated tests covering chess rules, game state, file I/O, tensor ops, model paths, Classical evaluation, and MCTS,
- a Linux build and a packaged Windows x64 build.

Current local verification on 2026-06-08 passes 151 tests with 0 failures.

3. Main Technical Decisions

The most important design decision was separating the headless core from the raylib UI. `cnv_core` contains chess, game state, file I/O, tensor operations, model loading, model evaluation, Classical evaluation, and MCTS. `cnv_viz` contains the board, controls, dialogs, activation panels, and tree drawing. This made automated testing practical and kept UI code from becoming the place where chess rules live.

The second major decision was to ship compact visualizer model files instead of huge upstream engine files. The CNN and BT4 defaults are small generated models, but they still run the real native LC0J/BT4J forward paths. This keeps the app portable enough for a course demo while preserving the architecture shapes that the visualizer is meant to explain.

The third major decision was to make Classical evaluation a first-class view rather than only a fallback. Its term breakdown and piece-square heatmaps give the user a concrete non-neural baseline beside NNUE, CNN, and BT4.

4. Biggest Challenges

Dense model data is hard to show in a readable way. A chess board has 64 squares, but the networks expose thousands of intermediate values. The final UI uses several levels of detail: board overlays, summary bars, node graphs, heatmaps, atlases, attention boards, policy bars, WDL readouts, and static architecture diagrams.

BT4 was also challenging because the original full model topology is too large for a lightweight visualizer. The solution was a compact BT4J v2 file and a native forward pass that supports the same style of token, attention, policy, and value records without shipping a hundreds-of-megabytes weight file.

Search visualization required another layer of state management. The MCTS engine must run independently from raylib, publish bounded snapshots, and allow the UI to pan, zoom, filter, merge transpositions, scrub growth frames, and trace selected nodes back into the activation views.

5. What Worked Well

The testable core/UI split worked well. It allowed the chess rules, model loaders, tensor operations, Classical evaluator, and MCTS engine to be verified headlessly. The UI then consumes stable snapshots and positions instead of owning rule logic.

The linked-list move history also worked well. It provides natural undo/redo, move-list rendering, and branch truncation when a new move is made after an undo.

The architecture-specific views make the app useful as a visual explanation tool: NNUE shows feature-to-accumulator flow, CNN shows spatial feature maps, BT4 shows attention over board tokens, and Classical shows explicit evaluator terms and PST heatmaps.

6. Remaining Limitations

- There is no engine-opponent play mode yet.
- FEN load/save is implemented, but PGN import and file pickers are not.
- The compact CNN and BT4 defaults are untrained visualization models, so they demonstrate architecture behavior rather than playing strength.
- Official LC0 protobuf model import is not implemented.
- UI testing remains mostly manual because the app is an interactive raylib desktop program.

7. Future Work

Useful extensions after the course submission would be:

- engine-vs-human mode using the existing search/evaluation path,
- PGN import and configurable save/load paths,
- side-by-side architecture comparison,
- activation difference mode between two positions,
- import support for official LC0 protobuf networks,
- automated screenshot or input-playback tests for the raylib UI.

8. Acknowledgements

The implementation was informed by prior personal chess projects:

- <https://github.com/LenniaConrad/chess-rtk>
- <https://github.com/LenniaConrad/chess-web>

Those projects were used as background knowledge and reference material. The submitted runtime code is C++ and does not depend on Java or TypeScript repositories at execution time.

The app was not made by AI. AI assistants were used only for minor support such as documentation wording, debugging explanations, and review/checklist help. The design, implementation, integration, and final testing remained student work.

9. AI Usage Summary

AI assistance was used as a support tool, not as the project author. The useful tasks were documentation cleanup, explaining compiler/build/debugging issues, reviewing whether documents matched the current source tree, and suggesting test/checklist items after the implementation changed.

AI output needed manual correction in several places. Model dimensions, BT4 block/head counts, UI behavior, and chess-rule edge cases had to be checked against the actual C++ source, runtime model files, raylib window, and automated tests. Suggestions that did not match the architecture were rewritten or removed.

The submitted application does not call AI services at runtime. It uses local C++ code and local binary model files only. Full details are recorded in `docs/ai-usage.md`.